

Discord Authenticator – Bericht

Joel Damm (jd106), Joschua Jung (jj050)

Inhalt

Worum geht es in diesem Projekt?.....	2
Funktionsweise.....	3
Projektverlauf.....	5
Schwierigkeiten und Lernerfahrungen.....	6
Fazit:.....	7

Worum geht es in diesem Projekt?

In diesem Softwareprojekt handelt es sich um einen Authentifizierungsdienst, der es Nutzern ermöglicht, sich auf Webseiten mithilfe eines Discord – Accounts anzumelden (ähnlich wie „Mit Google / Facebook anmelden“). Dies wurde mithilfe von OAuth 2 konzipiert.

Das Projekt beinhaltet:

- Reverse Proxy für einen Sicheren Zugriff
- Ein Frontend, entwickelt mit Svelte
- Backend, geschrieben in Rust
- PostgreSQL Datenbank für Nutzerdaten
- Redis Datenbank für Session-Verwaltung
- Containerisierung durch Docker

Funktionsweise

Authentifizierung mit OAuth2 funktioniert nach folgendem Prinzip:

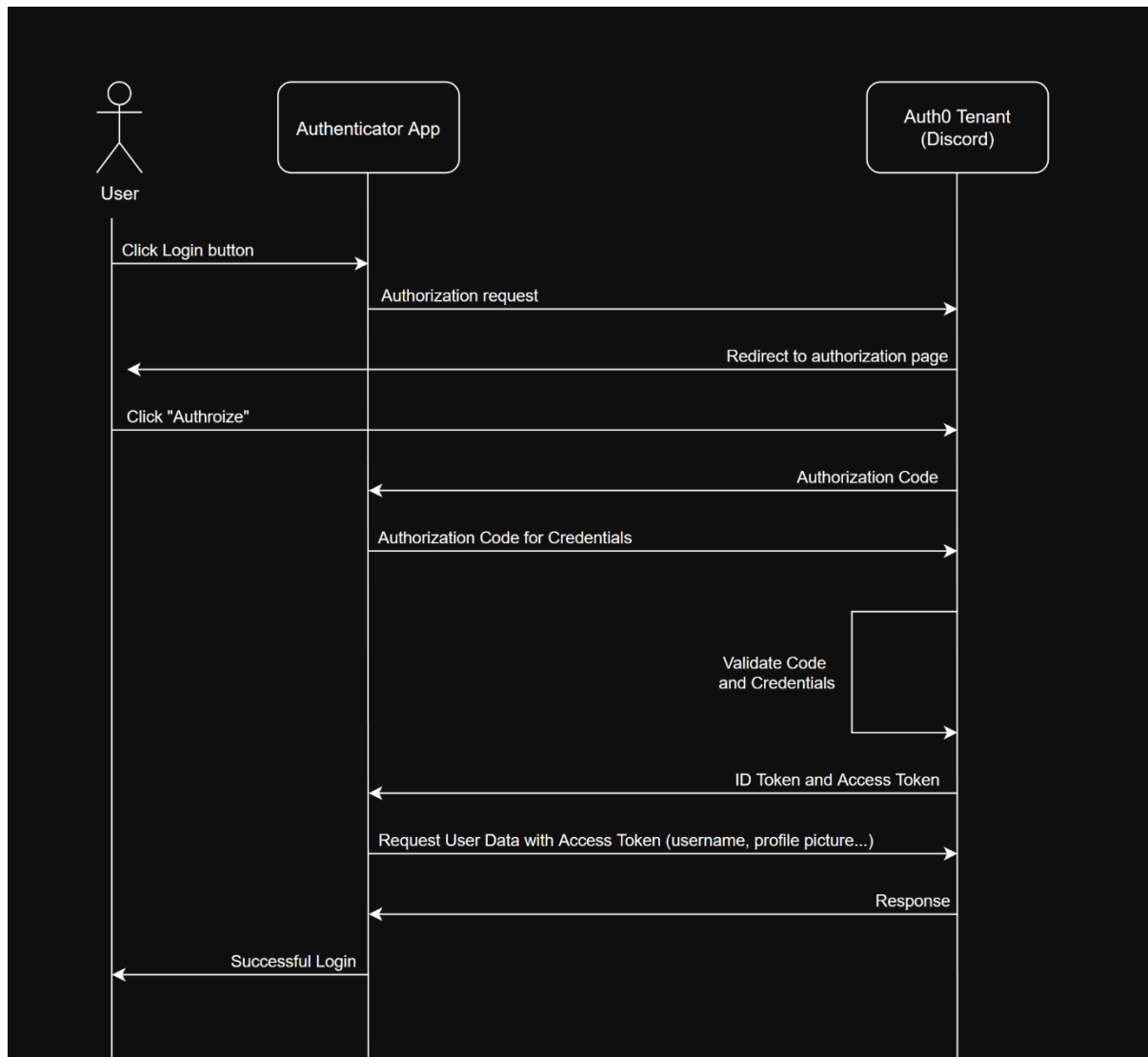


Abb.1: Vereinfachte Darstellung des OAuth2 Flow.

Es werden also keine sensiblen Daten wie z.B. Passwörter übertragen, die Authenticator App speichert lediglich ein Zugangs-Token, womit Informationen wie Benutzername und Profilbild von dem Tenant angefragt werden können.

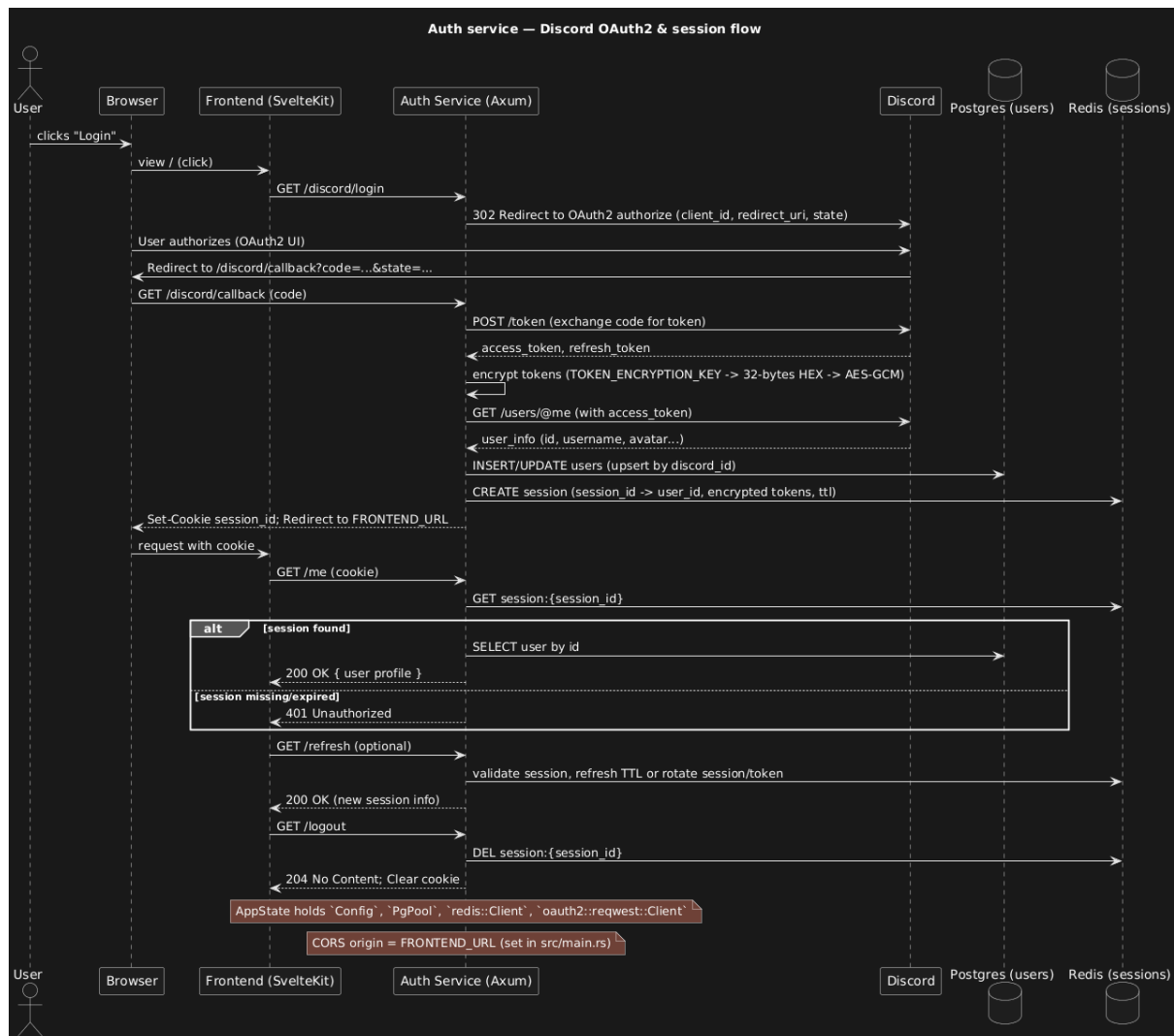


Abb.2: Sequenzdiagramm des Auth-Service, dargestellt sind Anmeldung und Abmeldung.

In unserem Authenticator wird das Access-Token verschlüsselt und in einer Redis Datenbank gespeichert, und die damit angefragten Daten (Benutzername, E-Mail und Profilbild) in einer PostgreSQL Datenbank.

Danach gibt der Auth Service ein session_id Cookie an das Frontend weiter, mit dem der Benutzer dann auf das Dashboard zugreifen kann.

Projektverlauf

Planung und Architektur

Ziele waren:

- sichere OAuth2 Implementierung (PKCE, CSRF-Schutz)
- skalierbare Session-Verwaltung (Redis)
- Type-Safe Datenbankoperationen (SQLx)
- ansprechende UI

Unser Stack wurde:

- Backend: Axum (Rust) für Performance, Type-Safety, Memory Safety
- DB: PostgreSQL und SQLx für compile time query validation
- Session Store: Redis für TTL (Time to live) support für die Sessions
- Frontend: Sveltekit für Reaktivität und Server Side Rendering und Session Validation
- Encryption via AES-256-GCM | Industrie Standard

Alles organisiert in einem Monorepo.

Gestartet haben wir mit unserem Backend, um alle CRUD Funktionen abzudecken und die User- und Sessionverwaltung zu implementieren, da diese die größte Herausforderung dargestellt hat. Hier wurde viel Zeit verbracht, da wir Rust lernen mussten sowie die verschiedenen Libraries, die wir benutzt haben, um mit der DB und Redis zu kommunizieren und die Cookies/Zertifikate zu verwalten.

Danach haben wir uns mit der Containerisierung und der Erstellung von Docker Images für unser Backend und unseren Frontend beschäftigt.

Um die Sicherheit zu gewährleisten, haben wir die Services hinter einem Reverse Proxy (Caddy) gesetzt, wodurch alle Anfragen mit Zertifikat und Cookies sicher über eine Domain Origin weitergeleitet werden.

Nun musste viel Troubleshooting betrieben werden, da das Weiterleiten von Cookies und den Zertifikaten viel Konfiguration erforderte innerhalb des Backends und des Frontends. Der erste Auth Flow war möglich, aber viele Bugs und die Integration der Discord API erwies sich sehr zeitintensiv.

Alles kam zusammen und mit dem ersten Frontend haben wir unseren Prototypen unserem Betreuer vorgeführt.

Nach Reflexion und Feedback wurde uns klar, dass wir an der Präsentation und Verständlichkeit unseres Projektes arbeiten mussten, um es für die Medianight präsentierbar zu machen. Somit haben wir das komplette Frontend überarbeitet und Dokumentation sowie Auth-Flow Visualisierungen ergänzt.

Schwierigkeiten und Lernerfahrungen

Die meisten Probleme wurden durch die hohe Komplexität des Projekts sowie unsere Unerfahrenheit mit dem Tech-Stack verursacht. Mit wachsender Erfahrung wurden diese aber nach und nach gelöst.

Frontend

Das Frontend wurde mit Svelte und Tailwind CSS gestaltet. Beim Erstellen des Webinterfaces kam es manchmal zu Verwirrungen mit der Vererbung von Elementen und bei der Strukturierung der Seiten. Jedoch wurden diese durch Recherche gelöst. Tailwind CSS verursachte Probleme beim Styling und Formatieren von Elementen, jedoch stammen die meisten von fehlendem Wissen und Erfahrung. Dies hat zur Folge, dass Tailwind CSS an einigen Stellen von gewöhnlichem CSS überschrieben wird.

Oftmals wurde während dem Aufsetzen statt des verwendeten pnpm versehentlich aus Gewohnheit npm installiert, was zu Dependency-Fehlern führte.

Backend

Wie bereits erwähnt, hat das Weiterleiten von Cookies und Zertifikaten viel Troubleshooting erfordert, da die Konfiguration sich als recht komplex erwies. Die Integration der Discord-API erwies sich als sehr zeitaufwändig.

Oftmals gab es Probleme mit Dependencies, die aus unbekannten Gründen nicht korrekt funktionierten. Diese Probleme wurden durch ein erneutes Aufsetzen des Projekts meistens gelöst. Der schlimmste Vorfall war mit cmake (wird für Rust benötigt) auf einem unserer Rechner, welcher dazu führte, dass das Betriebssystem komplett neu installiert werden musste. Ein weiterer Problempunkt war sqlx-cli, welches zum Migrieren von Daten verwendet wurde. Hier kam es oft zu Problemen mit unabsichtlich modifizierten Daten, was den Fehler "migration was previously applied but has been modified" auslöste.

Eine weitere Störung war der Zeitaufwand für das Bauen der Docker-Container. Diese mussten bei großen Änderungen und beim Neuaufsetzen neu gebaut werden, was jedes Mal mehrere Minuten kostete, umso mehr auf schwächeren Maschinen wie Laptops. Bei kleinen Änderungen musste das betroffene Container-Image neu aufgebaut werden, bevor die Änderungen sichtbar wurden, was sich im Zeitaufwand stark gehäuft hat.

Fazit:

Das Projekt war ein Full-Stack Learning Projekt, das Security, Database Design, OAuth2, und moderne Frontend-Architektur verband. Die größten Learnings:

1. Security muss von anfang an geplant werden — aber mit Env Vars und sicheren Libraries ist sichere Implementierung machbar
2. Compile-Time Checks sind wertvoll -> SQLx + TypeScript sparen Debug-Zeit
3. Configuration Management früh planen — Env Vars reduzieren Fehler
4. Standards nutzen — PKCE, AES-GCM, OAuth Flows sind bewährt

Das System ist ready, testbar und bietet einen Ansatz für eine sichere, skalierbare Basis, die mehrere Frontends und Services unterstützt (bisher nur im localhost getestet).